

Exercise 1: Forward & Reverse Record Queries

1.1: Query IPv4 Mapping (A Record) and Subdomain Canonical Names (CNAME)

In this step, I executed a forward DNS lookup using the `dig wikipedia.org A` command to retrieve the IPv4 address. I also tested the `+short` flag for concise output and looked up a CNAME record for GitHub to see how aliases are handled.

```
Last login: Mon Aug 17 21:18:13 on console
[(base) viraj@Virajs-MacBook-Pro ~ % dig -v
DiG 9.10.6
[(base) viraj@Virajs-MacBook-Pro ~ % dig wikipedia.org A

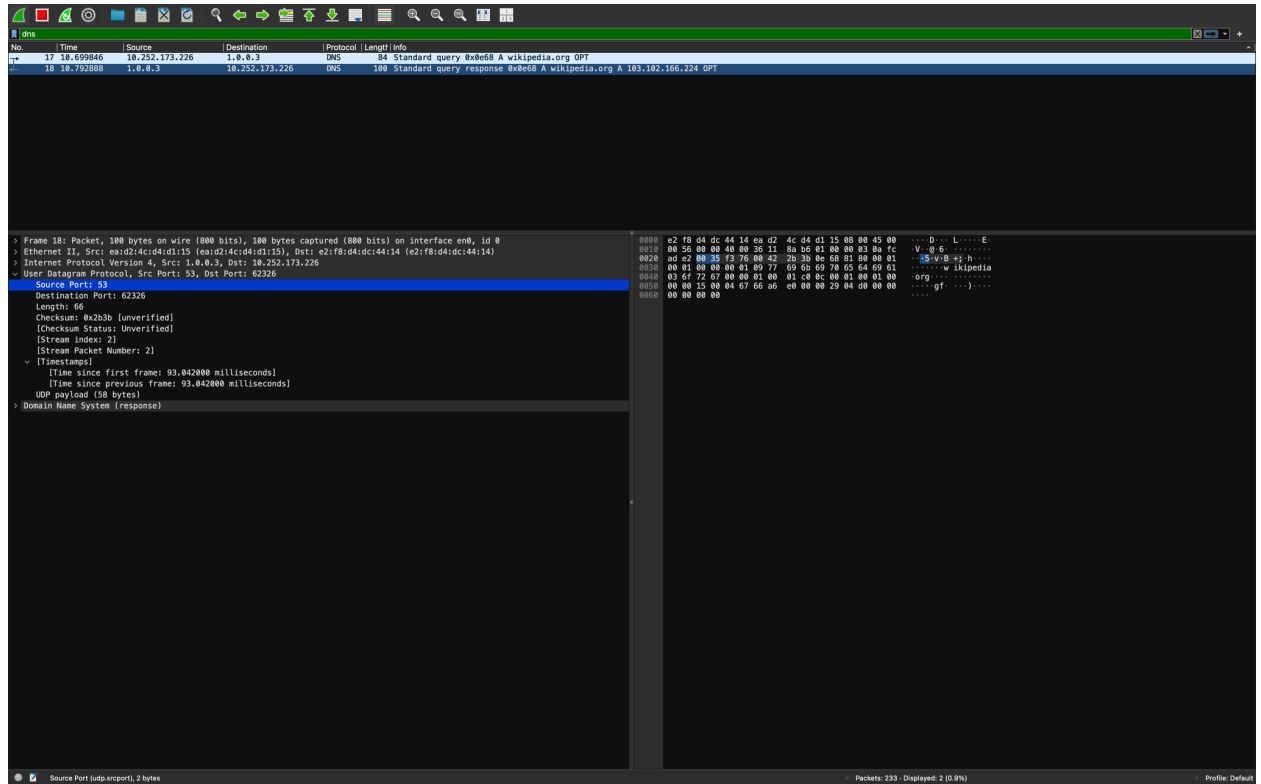
; <<>> DiG 9.10.6 <<>> wikipedia.org A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 63284
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;wikipedia.org.                IN      A

;; ANSWER SECTION:
wikipedia.org.                180     IN      A      103.102.166.224

;; Query time: 146 msec
;; SERVER: 1.0.0.3#53(1.0.0.3)
;; WHEN: Mon Aug 17 22:11:50 IST 2026
;; MSG SIZE rcvd: 58

(base) viraj@Virajs-MacBook-Pro ~ %
```



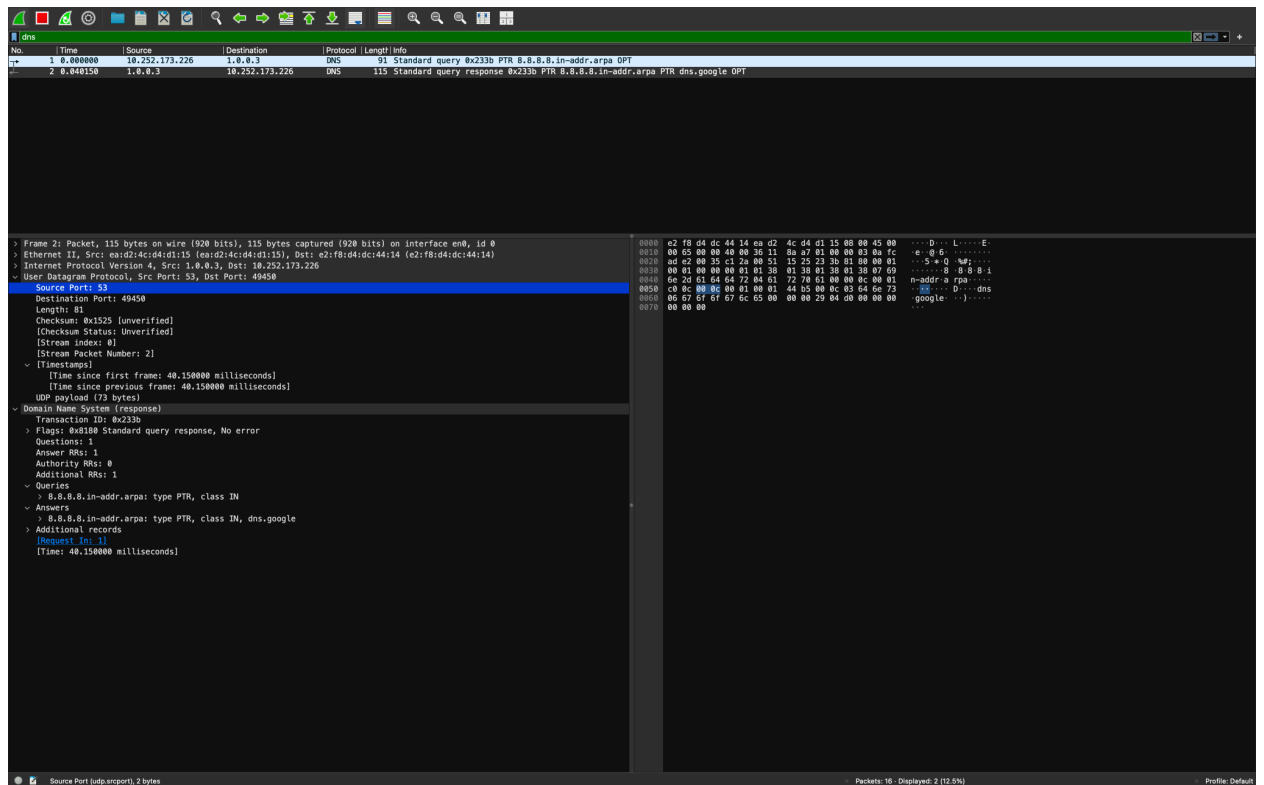
[Terminal showing 'dig wikipedia.org A' and Wireshark capture]

Observations: By analyzing the ANSWER SECTION in the output, the IP address returned for wikipedia.org was 103.102.166.224. For the GitHub query, the output clearly demonstrates how www.github.com acts as an alias pointing to the actual underlying target hostname.

1.2: Reverse IP Address Lookup (PTR Record)

Next, I used the `dig -x 8.8.8.8 +short` command to perform a reverse DNS lookup, aiming to find the hostname associated with Google's public IP address.

```
[(base) viraj@Virajs-MacBook-Pro ~ % dig -x 8.8.8.8 +short  
dns.google.  
(base) viraj@Virajs-MacBook-Pro ~ %
```



[Terminal showing 'dig -x 8.8.8.8 +short']

Observations: The reverse lookup successfully returned dns.google., confirming that PTR records correctly map the IP address back to its human-readable domain name.

Exercise 2: Tracing Iterative Resolution Hierarchy

2.1: Perform an Iterative Trace

To observe how a DNS query travels through the internet's hierarchy, I ran the trace command: `dig aws.amazon.com +trace`.

```
(base) viraj@Viraj-s-MacBook-Pro ~ % dig aws.amazon.com +trace

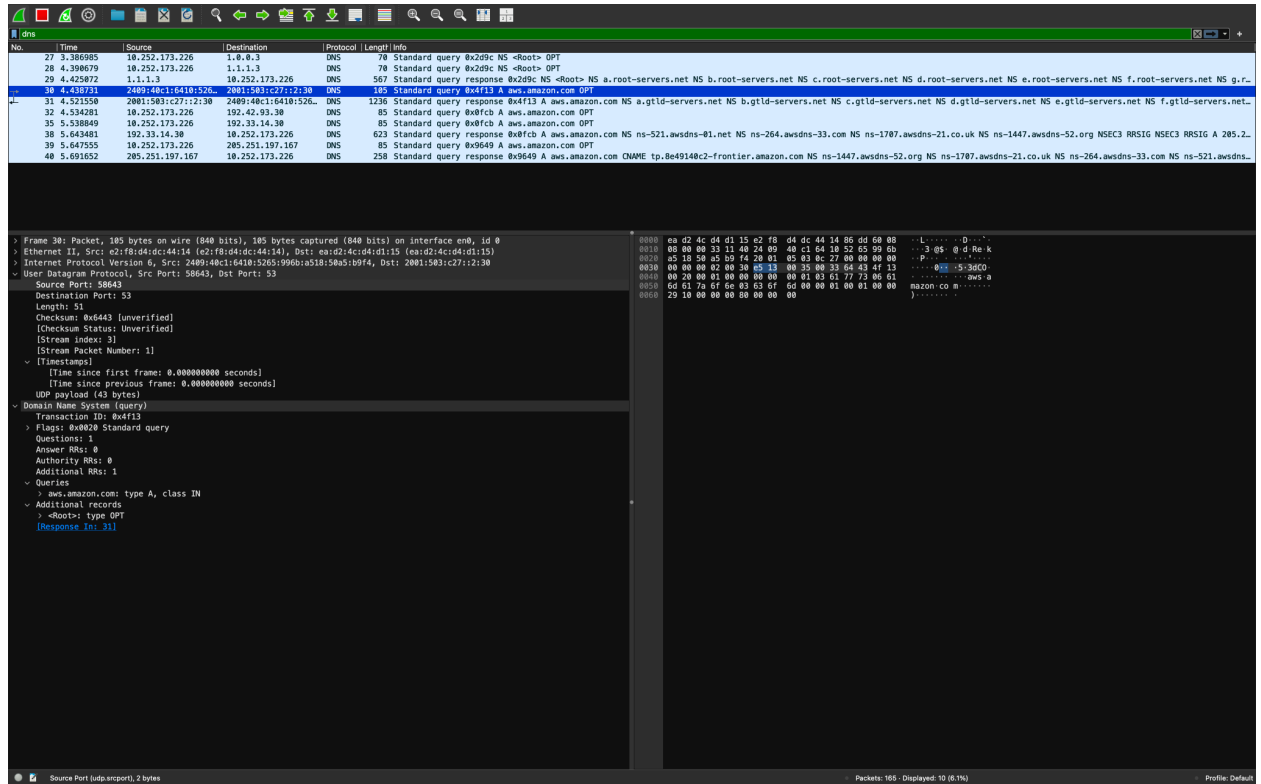
;<<> Dig 9.18.6 <<> aws.amazon.com +trace
;; global options: +cmd
51787A IN NS a.root-servers.net.
51787A IN NS b.root-servers.net.
51787A IN NS c.root-servers.net.
51787A IN NS d.root-servers.net.
51787A IN NS e.root-servers.net.
51787A IN NS f.root-servers.net.
51787A IN NS g.root-servers.net.
51787A IN NS h.root-servers.net.
51787A IN NS i.root-servers.net.
51787A IN NS j.root-servers.net.
51787A IN NS k.root-servers.net.
51787A IN NS l.root-servers.net.
51787A IN NS m.root-servers.net.
51787A IN RRSIG NS 8 528400 20240805080800 20240817040800 57788 - 3zn4k4y+eS3YvZw4N6oKtIF/QdIC-3tHLQPMD7KnpKamIP2Paw/S x81w8BHdEpESFQXJ8R1o2d4dM6rj+chvYzFmTLr832r189N87x F5CTV20Mgy381/aCfKac294odBsk/gmbpmaKtYAHJ5awb1e78Bn2oA HEdL2xLFY828h7qI/a
TBaFJa/Plw7aMgJ-8rIVQAAhRgPzKxAV1mb- NtUp4M8EFT1a5/3U/a331P17/0BVawZudfTnwoqt1oFtVQqicInfx ym28rFolEu0Uw71K3FFGCBV8PcB10KDDckJf+akQzpxXZ0uakvS Ios54q==
;; Received 826 bytes from 194.0.36.51 in 38 ms

com. 172880 IN NS a.g11d-servers.net.
com. 172880 IN NS b.g11d-servers.net.
com. 172880 IN NS c.g11d-servers.net.
com. 172880 IN NS d.g11d-servers.net.
com. 172880 IN NS e.g11d-servers.net.
com. 172880 IN NS f.g11d-servers.net.
com. 172880 IN NS g.g11d-servers.net.
com. 172880 IN NS h.g11d-servers.net.
com. 172880 IN NS i.g11d-servers.net.
com. 172880 IN NS j.g11d-servers.net.
com. 172880 IN NS k.g11d-servers.net.
com. 172880 IN NS l.g11d-servers.net.
com. 172880 IN NS m.g11d-servers.net.
com. 86400 IN RRSIG DS 8 1 86400 20240818080800 20240817040800 57788 - 1a23ug7FFFQW7/cexX2R18uJ0wDVMgH9x1ldPeagQ208rFscHFA 1l35gVp/rrQh75y9vAFJ8CDBV8dcCOPQ8HmHfWuQz7ZhuZ9RXj3j 1t1uYoInFkJulnSdsC8X/BcaahgbcU/aMeS33u/MwmmhgyHYIOL Qo1YKXPr47LKQJjEars
BwH8C8u2ARICFmwhbFt5ymEVC1a6G0DP v6f8u6Z1aQW jx1Lw1R20v3Yw+HPPAZDTVO10Jr8auA8aK2L3Jqz e+shnfz1Z2p8p0mNLV21MKSdMgqz70HW/Xpuc32c38PTED8FQdow +mQAg==
;; Received 1174 bytes from 20851:7ff1:1853:k.root-servers.net in 58 ms

amazon.com. 172880 IN NS ns-521.awdns-81.net.
amazon.com. 172880 IN NS ns-264.awdns-53.com.
amazon.com. 172880 IN NS ns-1707.awdns-21.co.uk.
amazon.com. 172880 IN NS ns-1447.awdns-52.org.
Ck8PQMG67AL3RE7EFNA3830V1888.com. 980 IN RRSIG NSC3 1 3 8 - Ck8PQMG67AL3RE7EFNA3830V1888.com. NS SDA RRSIG DNSKEY NSC3PARAM
Ck8PQMG67AL3RE7EFNA3830V1888.com. 980 IN RRSIG NSC3 1 3 2 988 20240822020615 20240814231615 45446 com. GR23205xXTCz8yHjRm/N2wWjy4G1+MDjgeUwWGSF+uPVC2agZULK DzYux3b26RPcwh+16MnzCu7a8X/C2Q==
K38W84125A8M8933227LVKACAJURD.com. 980 IN NSC3 1 3 8 - K38W84125A8M8933227LVKACAJURD.com. NS DS RRSIG
K38W84125A8M8933227LVKACAJURD.com. 980 IN RRSIG NSC3 1 3 2 988 20240822020615 20240815080642 45446 com. ThRG7eroolV34GcW5nZyPzQZu4w98Ljuu+JpuJ8KfagqrRWTTo nfJnTQyK1A088+rzskYTQuzr8Aa==
;; Received 881 bytes from 192.0.36.51 in 148 ms

aws.amazon.com. 7200 IN CNAME tp.8a6t8148c3-frontlist.amazonaws.com.
amazon.com. 7200 IN NS ns-1447.awdns-52.org.
amazon.com. 7200 IN NS ns-1707.awdns-21.co.uk.
amazon.com. 7200 IN NS ns-264.awdns-53.com.
amazon.com. 7200 IN NS ns-521.awdns-81.net.
;; Received 216 bytes from 2085.201.197.107853(ns-1447.awdns-52.org) in 83 ms

(base) viraj@Viraj-s-MacBook-Pro ~ %
```



[Terminal showing 'dig aws.amazon.com +trace' and Wireshark DNS trace]

Observations: The trace output clearly displayed the sequential stages of resolution. It first queried the Root Servers (.), then moved to the .com Top-Level Domain servers, located the authoritative servers for amazon.com, and finally retrieved the specific record for aws.amazon.com.

Exercise 3: TTL Caching Mechanics and Local Resolver Overrides

3.1: Observe TTL Decrement in Resolver Cache

I ran `dig example.com`, noted the Time-To-Live (TTL) value, and then immediately executed the exact same command again to observe caching behavior.

```
((base) viraj@Virajs-MacBook-Pro ~ % dig example.com

; <<>> DiG 9.10.6 <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 25856
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 168     IN      A      104.20.23.154
example.com.                 168     IN      A      172.66.147.243

;; Query time: 73 msec
;; SERVER: 1.0.0.3#53(1.0.0.3)
;; WHEN: Mon Aug 17 17:41:18 IST 2026
;; MSG SIZE rcvd: 72

((base) viraj@Virajs-MacBook-Pro ~ % dig example.com

; <<>> DiG 9.10.6 <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62731
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 60      IN      A      172.66.147.243
example.com.                 60      IN      A      104.20.23.154

;; Query time: 86 msec
;; SERVER: 1.0.0.3#53(1.0.0.3)
;; WHEN: Mon Aug 17 17:43:06 IST 2026
;; MSG SIZE rcvd: 72

(base) viraj@Virajs-MacBook-Pro ~ % █
```

The image displays two consecutive network packets captured by Wireshark, both related to a DNS query for 'example.com'.

Packet 13837 (Standard query):

- Time: 55.668285
- Source: 10.280.17.113
- Destination: 1.0.0.3
- Protocol: DNS
- Length: 82
- Info: Standard query 0x6500 A example.com OPT

Packet 38979 (Standard query response):

- Time: 181.684549
- Source: 10.280.17.113
- Destination: 1.0.0.3
- Protocol: DNS
- Length: 81
- Info: Standard query response 0x8f9a gateway.grammaly.com

The packet details for the response (38979) show the following structure:

- Frame 38979: Packet, 114 bytes on wire (912 bits), 114 bytes captured (912 bits) on interface en0, id 0
- Ethernet II, Src: Cisco_Se54:df (bc:b3:0e:5e:54:df), Dst: Se:e5:5b:bb:11:55 (Se:e5:5b:bb:11:55)
- Internet Protocol Version 4, Src: 1.0.0.3, Dst: 10.280.17.113
- User Datagram Protocol, Src Port: 53, Dst Port: 51640
- Domain Name System (response)
 - Transaction ID: 0x6500
 - Flags: 0x01aa Standard query response, No error
 - Questions: 1
 - Answer RRs: 2
 - Authority RRs: 0
 - Additional RRs: 1
- Queries
 - example.com: type A, class IN, addr 104.20.23.154
 - Name: example.com
 - Type: A (1) (Host Address)
 - Class: IN (0x0001)
 - Time to live: 168 (2 minutes, 48 seconds)
 - Data length: 4
 - Address: 104.20.23.154
 - example.com: type A, class IN, addr 172.66.147.243
 - Name: example.com
 - Type: A (1) (Host Address)
 - Class: IN (0x0001)
 - Time to live: 168 (2 minutes, 48 seconds)
 - Data length: 4
 - Address: 172.66.147.243
- Additional records
 - <Root>: type OPT
 - Name: <Root>
 - Type: OPT (41)
 - UDP payload size: 1232
 - Higher bits in extended RCODE: 0x00
 - EDNS0 version: 0
 - Z: 0x0000
 - 0..... = DO bit: Cannot handle DNSSEC security RRs
 - .000 0000 0000 0000 = Reserved: 0x0000
 - Data length: 0

The packet bytes section shows the raw data in hexadecimal and ASCII format.

[Terminal showing two consecutive 'dig example.com' commands]

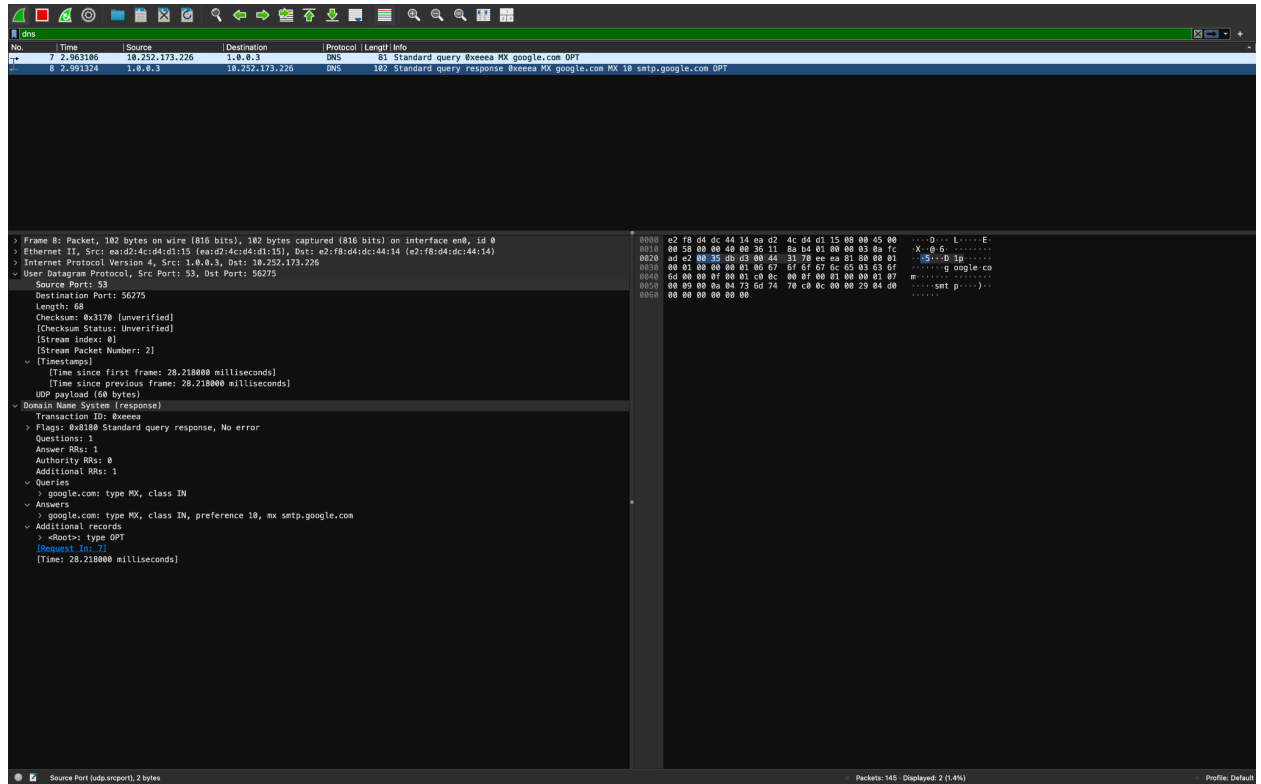
Observations: In the first query, the TTL showed a specific integer value. When I ran it the second time, the TTL number had noticeably decreased. This confirms that my local/upstream DNS resolver was serving the second request directly from its cache instead of reaching out to the authoritative server again.

Exercise 4: Mail Exchange (MX) and Text (TXT) Metadata Records

4.1 & 4.2: Query MX and TXT Records

To inspect secondary DNS records, I queried the mail servers for Google using `dig google.com MX +short` and checked the text records using `dig google.com TXT +short`.

```
[(base) viraj@Virajs-MacBook-Pro ~ % dig google.com MX +short
10 smtp.google.com.
(base) viraj@Virajs-MacBook-Pro ~ % █
```



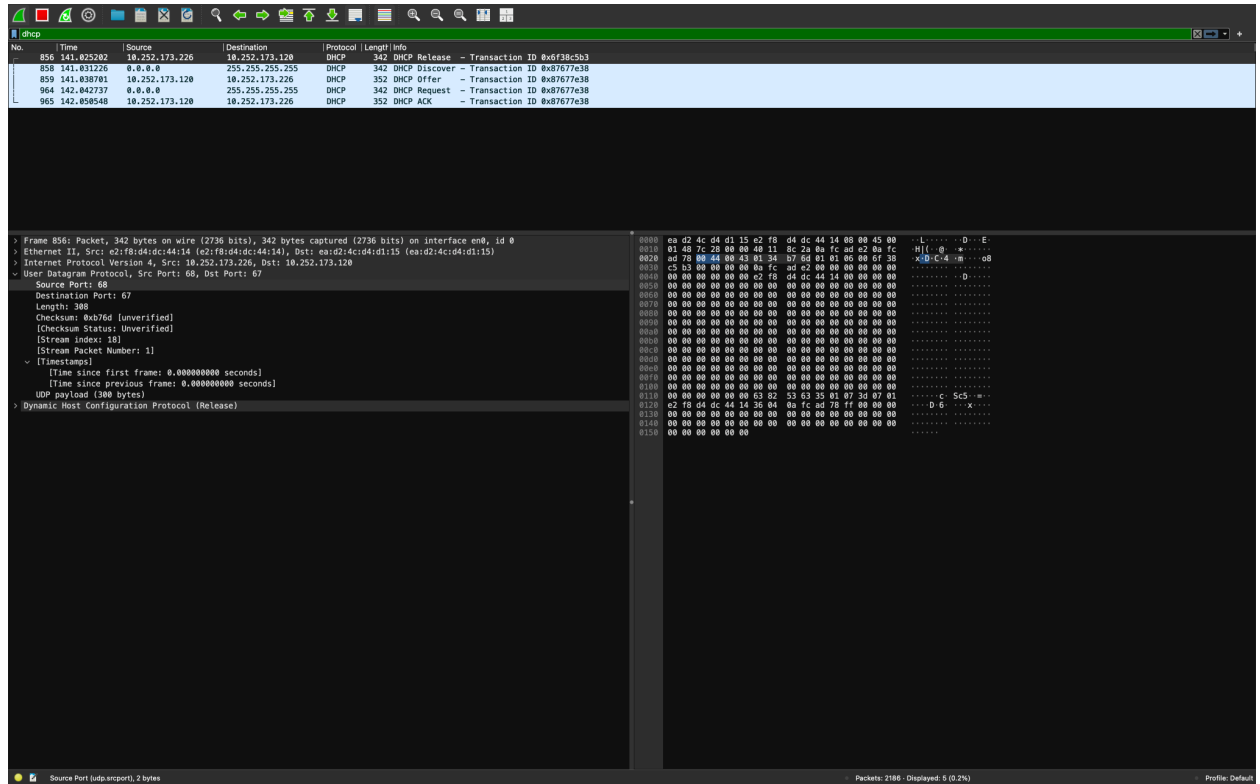
[Terminal showing 'dig google.com MX +short' and TXT queries]

Observations: The MX output showed records like 10 smtp.google.com., highlighting the mail server address alongside its preference priority number (10). The TXT output revealed various verification strings and SPF policies used for domain security.

Exercise 5: DHCP

To observe how IP addresses are dynamically assigned on my network, I used the Mac terminal commands to request a new IP via DHCP while capturing the traffic in Wireshark.

```
[(base) viraj@Virajs-MacBook-Pro ~ % sudo ipconfig set en0 DHCP
[Password:
(base) viraj@Virajs-MacBook-Pro ~ %
```



[terminal running 'sudo ipconfig set en0 DHCP' and Wireshark DORA process]

Observations: The Wireshark capture successfully documented the DORA process (DHCP Discover, Offer, Request, ACK) occurring in the background as the system negotiated a new IP lease with the local router.

3. Lab Assessment Questions

Section A: Output Analysis

- A network engineer executes `dig dev.example.com` and receives **NXDOMAIN** in the header section. What does this response code indicate?

This basically means the domain doesn't exist. "NXDOMAIN" stands for Non-Existent Domain. So, the DNS server is letting you know that it checked the registry but couldn't find any records associated with `dev.example.com`. It's similar to getting a 404 error on a website, but at the network level.

- If an MX record query returns both 10 mail1.example.com and 20 mail2.example.com, which server will a sending mail transfer agent (MTA)

attempt to deliver to first? Explain why.

The mail server will try to send the email to mail1.example.com first. This is because MX records use preference numbers to determine priority, and the rule is that the lower the number, the higher the priority. Since 10 is lower than 20, the system will always attempt delivery to that server first and will only fall back to the '20' server if the first one fails or is unavailable.

- **Why does running ping app.local use the /etc/hosts file entry, whereas dig app.local ignores it?**

Commands like ping rely on the operating system's built-in name resolution process. This process is designed to always check local manual overrides (like the /etc/hosts file) before it goes out to ask the internet's DNS servers. On the other hand, dig is a specialized networking tool made specifically to query DNS servers directly. It intentionally bypasses local host files so you can see exactly what the actual remote DNS server is reporting.

Section B: Practical Troubleshooting Scenario

- **What is causing the discrepancy between the two branch offices?**

This is a classic DNS caching issue. The DNS resolver at Branch Office B still has the old IP address (192.0.2.10) temporarily saved in its cache. Because the TTL (Time-To-Live) was set to a long duration of 86400 seconds, the cache hasn't expired yet for that specific branch. Branch Office A likely had an already-expired cache or queried the domain for the first time, allowing it to fetch the updated IP immediately.

- **How many hours will Branch Office B take to update automatically?**

It will take up to 24 hours. The TTL was set to 86400 seconds. If you divide 86400 by 3600 (the number of seconds in one hour), it perfectly equals 24 hours before the cache is forced to drop the old record and fetch the new one.

- **Which dig command can an engineer run to bypass local caching and query the domain's authoritative nameserver directly?**

An engineer can query the authoritative server directly by using the @ symbol followed by the nameserver's address, like this: dig @ns1.authoritative.com example.com. Alternatively, they can use the trace command (dig example.com +trace) to force the resolution to go all the way down the hierarchy from the root servers, bypassing any middleman caches.